Benchmarks

Benchmarks

`gemstone-js` includes a small maintained benchmark lane plus local benchmark artifact tooling for CI workflows. The `gci` suite is offline and needs no GemStone credentials. The persistence suites connect to a configured Stone and should be run intentionally against a disposable or benchmark image.

Benchmark reports use the same compact shape as gemstone-py artifacts:

```
{
  "$schema": "./schemas/benchmark-report.schema.json",
  "schema_version": 1,
  "stone": "gs64stone",
  "platform": "darwin-arm64",
  "runtime": "node",
  "node_version": "24.0.0",
  "gci_backend": "native",
  "entries": 500,
  "search_runs": 20,
  "suites": ["gci"],
  "results": [
    {
      "suite": "gci",
      "operation": "smallint_roundtrip",
      "elapsed_seconds": 0.016,
      "ops_per_second": 1200,
      "count": 20
    }
  ]
}
```

The package ships JSON Schemas for editor and CI validation:

- `schemas/benchmark-report.schema.json`
- `schemas/benchmark-baseline-manifest.schema.json`

Generated reports include `$schema`; baseline manifests created or updated by `gemstone-js-benchmark-register` include `$schema` as well.

Generate reports with `gemstone-js-benchmarks`:

```
npm run benchmarks -- --suite gci --entries 100000 --json --output benchmark-report.json
npm run benchmarks -- --suite persistent_root --suite gstore --entries 500 --json --output live-report.jso
npm run benchmark:validate -- benchmark-report.json
```

The default suite list is `gci`, `persistent_root`, `gscollection`, `gstore`, and `rchash`. Select only `gci` for a no-credential local check. Select any of the persistence suites only when `GS_*` connection environment variables point at a Stone prepared for benchmark data.

Compare a candidate report with a committed baseline:

```
npm run benchmark:compare -- baseline.json candidate.json
npm run benchmark:compare -- baseline.json candidate.json --max-regression-pct 10
npm run benchmark:compare -- candidate.json --manifest .github/benchmarks/index.json --max-regression-pct
npm run benchmark:compare -- baseline.json candidate.json --json --output benchmark-compare.json
```

When `--manifest` is supplied, comparison selects the single committed baseline whose comparable metadata matches the candidate report before applying thresholds. This is the shortest CI path when the manifest is kept free of duplicate metadata.

Select a committed baseline whose metadata matches a candidate:

```
npm run benchmark:validate -- benchmark-report.json --manifest .github/benchmarks/index.json
npm run benchmark:baselines -- benchmark-report.json --manifest .github/benchmarks/index.json
```

Manifest validation loads referenced baseline reports by default and rejects duplicate baseline metadata, because duplicate environment matches make baseline selection ambiguous. Use `--allow-duplicate-metadata` only for manual maintenance work where duplicates are intentional.

Register or maintain committed baseline artifacts:

```
npm run benchmark:register -- benchmark-report.json --manifest .github/benchmarks/index.json
npm run benchmark:register -- benchmark-report.json --copy-to baseline-macos-arm64.json
npm run benchmark:register -- benchmark-report.json --copy-to baseline-macos-arm64.json --replace-duplicat
npm run benchmark:register -- --manifest .github/benchmarks/index.json --prune-missing
```

Registration rejects a new baseline when another manifest entry already has the same comparable metadata. Use `--replace-duplicate-metadata` to rotate the manifest entry for that environment to the newly registered artifact. Use `--allow-duplicate-metadata` only when intentionally keeping multiple baselines for the same environment.

The comparison command supports global, suite-level, and operation-level regression thresholds. Operation thresholds take precedence over suite thresholds, and suite thresholds take precedence over the global threshold.